

Princípios de Programação

Exercícios

Universidade de Lisboa
Faculdade de Ciências
Departamento de Informática
Licenciatura em Engenharia Informática

2023/2024

Construção de tipos e de classes de tipos

1. Defina um tipo de dados que descreva as seguintes formas geométricas: círculo, rectângulo e triângulo. Escreva funções para calcular o perímetro e para verificar se uma figura é regular (uma forma é regular se todos os seus ângulos são iguais e todos os lados são iguais).

2. Considerando o tipo de dados `Nat`:

```
data Nat = Zero | Succ Nat
```

escreva as seguintes funções:

- (a) `add :: Nat -> Nat -> Nat`, a soma de dois naturais,
- (b) `monus :: Nat -> Nat -> Nat`, a subtração natural, i.e., se o 2º natural for maior que o 1º, a função é igual a zero,
- (c) `pred :: Nat -> Nat`, calcule o predecessor do natural dado. Esta função deverá estar indefinida para o natural zero,
- (d) `sub :: Nat -> Nat -> Nat`, a diferença de dois naturais. Use a função `pred`. Esta função deverá estar indefinida para valores do primeiro argumento menores que o segundo argumento,
- (e) `mult :: Nat -> Nat -> Nat`, o produto de dois naturais. Utilize a função `add`,
- (f) `pot :: Nat -> Nat -> Nat`, a potência do 1º natural elevado ao 2º natural,
- (g) `fact :: Nat -> Nat`, o factorial do natural dado,
- (h) `lessThan :: Nat -> Nat -> Bool`, verifica se o 1º natural é menor que o 2º natural,

- (i) `remnat :: Nat -> Nat -> Nat`, o resto da divisão inteira entre o 1º e o 2º natural,
 - (j) `quotnat :: Nat -> Nat -> Nat`, o quociente da divisão inteira entre o 1º e o 2º natural.
3. Para o exercício sobre conjuntos da secção anterior, escolha um tipo de dados mais apropriado, um tipo de dados que melhore a complexidade (o o-grande, $\mathcal{O}$) das várias operações. Reescreva depois o módulo de modo a esconder a estrutura do tipo de dados.
 4. Para o exercício sobre mapas da secção anterior, escreva um tipo de dados **Map** mais apropriado, um tipo de dados que melhore a complexidade (o o-grande, $\mathcal{O}$) das várias operações. Reescreva depois o módulo de modo a esconder a estrutura do tipo de dados.
 5. Considere o tipo de dados *árvores de pesquisa*.

data Arvore a = Folha | No (Arvore a) a (Arvore a)

Assuma que numa árvore de pesquisa, os valores estão ordenados: o valor de cada nó é maior ou igual a cada valor da sub-árvore esquerda, e o valor de cada nó é menor ou igual que cada valor da sub-árvore direita. Escreva as funções abaixo.

- (a) `empty :: Arvore a`, uma árvore vazia.
- (b) `singleton :: a -> Arvore a`, uma árvore com um elemento.
- (c) `size :: Arvore a -> Int`, o número de nós na árvore.
- (d) `depth :: Arvore a -> Int`, a profundidade da árvore. A profundidade de uma árvore vazia é zero; aquela de uma árvore não vazia é um mais o máximo das profundidades das sub-árvores.
- (e) `isPerfect :: Arvore a -> Bool`, a árvore é perfeita? Uma árvore vazia é considerada perfeita. Uma árvore não vazia diz-se perfeita se as duas sub-árvores são perfeitas e têm o mesmo número de nós. Numa primeira fase, resolva este exercício recorrendo à função `size`. Analise a sua complexidade. Desenhe depois uma solução que não percorra a árvore mais do que uma vez.
- (f) `insert :: Ord a => a -> Arvore a -> Arvore a`, insere um elemento numa árvore de pesquisa (mantendo a ordenação).
- (g) `fromList :: Ord a => [a] -> Arvore a`, uma árvore sintetizada a partir de uma lista de elementos (não ordenada).
- (h) `toList :: Arvore a -> [a]`, a lista dos elementos da árvore visitados pelo percurso infixo. O percurso infixo de uma árvore não vazia visita primeiro a sub-árvore esquerda, depois o elemento do nó e finalmente a sub-árvore direita.

- (i) `elem :: Ord a => a -> Arvore a -> Bool`, verifica se um dado elemento consta de uma árvore.
 - (j) `allIn :: Ord a => Arvore a -> Arvore a -> Bool`, verifica se todos os elementos de uma dada árvore constam de uma outra árvore.
6. Reescreva as alíneas c, d, e, h, i, j do exercício 5 recorrendo à seguinte função `fold`.
- ```
fold :: (b -> a -> b -> b) -> b -> Arvore a -> b
fold _ e Folha = e
fold f e (No l x r) = f (fold f e l) x (fold f e r)
```
7. Torne o tipo de dados `Arvore a` instância da classe `Eq`. Para efeitos deste exercício duas árvores são iguais se contiverem os mesmos elementos.
8. Torne o tipo de dados `Arvore a` instância da classe `Show`. A conversão de uma árvore numa `String` deverá ser tal que a árvore

```
No (No Folha "cao" (No Folha "gato"
 Folha)) "peixe" (No Folha "pulga" Folha)
```

seja convertida em

```
"peixe"
"cao"
 Folha
 "gato"
 Folha
 Folha
 "pulga"
 Folha
 Folha
```

9. A a classe de tipos `Foldable t` aplica-se a todos os tipos que possam ser percorridos na sua totalidade, para produzir um dado valor. Para tornar o tipo `Arvore` (definida no exercício 5) instância da classe de tipos `Foldable` basta escrever uma função com a seguinte assinatura.

```
foldr :: (a -> b -> b) -> b -> Arvore a -> b
```

Depois ganhamos automaticamente uma série de funções derivadas do `foldr`, incluindo

```
foldl :: (b -> a -> b) -> b -> Arvore a -> b
foldr1 :: (a -> a -> a) -> Arvore a -> a
foldl1 :: (a -> a -> a) -> Arvore a -> a
null :: Arvore a -> Bool
```

```
length :: Arvore a -> Int
elem :: Eq a => a -> Arvore a -> Bool
maximum :: Ord a => Arvore a -> a
minimum :: Ord a => Arvore a -> a
sum :: Num a => Arvore a -> a
product :: Num a => Arvore a -> a
```

- (a) Torne o tipo de dados `Arvore` instância da classe de tipos `Foldable`. Para isso basta escrever uma função com a seguinte assinatura.

```
foldr :: (a -> b -> b) -> b -> Arvore a -> b
```

A função deverá percorrer a árvore no sentido infixo-inverso, isto é: primeiro percorre a subárvore direita, depois a raiz e finalmente a subárvore esquerda. Por exemplo, usando a função `fromList` do exercício 5 temos:

```
> t = fromList [3, 6, 8, 0, 7, 9, 2]
> foldr (:) [] t
[0,2,3,6,7,8,9]
> foldl (flip (:)) [] t
[9,8,7,6,3,2,0]
> sum t
35
> maximum t
9
```

- (b) Reescreva a função `length` usando a função `foldr`.  
 (c) Reescreva a função `elem` usando a função `foldr`.  
 (d) Reescreva a função `sum` usando a função `foldr`.  
 (e) Escreva uma função `toListR :: Arvore a -> [a]` que devolve a lista de elementos na árvore pelo sentido infixo-inverso.  
 (f) Escreva uma função `toListL :: Arvore a -> [a]` que devolve a lista de elementos na árvore pelo sentido infixo convencional.
10. Torne o tipo `Arvore` instância da class `Functor`. Exemplo:

```
*Set> fmap (^2) (Node (Node EmptyTree 4 EmptyTree) 5
(Node EmptyTree 6 EmptyTree))
25
16
Empty
Empty
36
Empty
Empty
```

11. Torne o tipo `Set` do capítulo anterior instância das classes **`Eq`** e **`Show`**. No caso de **`Show`** represente o conjunto com elementos separados por vírgulas e entre parêntesis, como é habitual em matemática.

```
*Set> fromList [1,5..30]
{1,5,9,13,17,21,25,29}
```

12. Torne o tipo `Set` do capítulo anterior instância da class **`Functor`**. Exemplo:

```
*Set> fmap (*2) (fromList [1,5..30])
{2,10,18,26,34,42,50,58}
```

13. Considere o tipo de dados

```
data Map k a = M [(k, a)]
```

que representa um mapa baseado em *listas de associação*, isto é, através de listas de pares chave-valor. Considere que esta representação mantém duas invariantes:

- (a) As chaves não aparecem repetidas,
- (b) A lista está ordenada pelas chaves.

Torne o tipo **`Map`** instância da classe **`Eq`**. Dois mapas são iguais se contiverem as mesmas chaves, e para cada chave apresentarem valores iguais.

14. Torne o tipo **`Map`** do exercício anterior instância da classe **`Show`**. Ao mapa com duas entradas `("a", 1)` e `("b", 2)` deve corresponder a `string {"a": 1, "b": 2}`.

15. Considere a classe `Visible` definida da seguinte forma:

```
class Visible a where
 toString :: a -> String
 dimension :: a -> Int
```

Crie instâncias desta classe para os tipos **`Char`**, **`Bool`**, lista de `Visible` e pares de `Visible`.

16. Complete as seguintes declarações.

```
instance (Ord a, Ord b) => Ord (a,b) where ...
instance Ord a => Ord [a] where ...
```